

T16 — Java Collections Framework (JCF)

Gerarchia Collection, List, Set, Map, Queue; Prestazioni e Scelta delle Strutture Dati

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

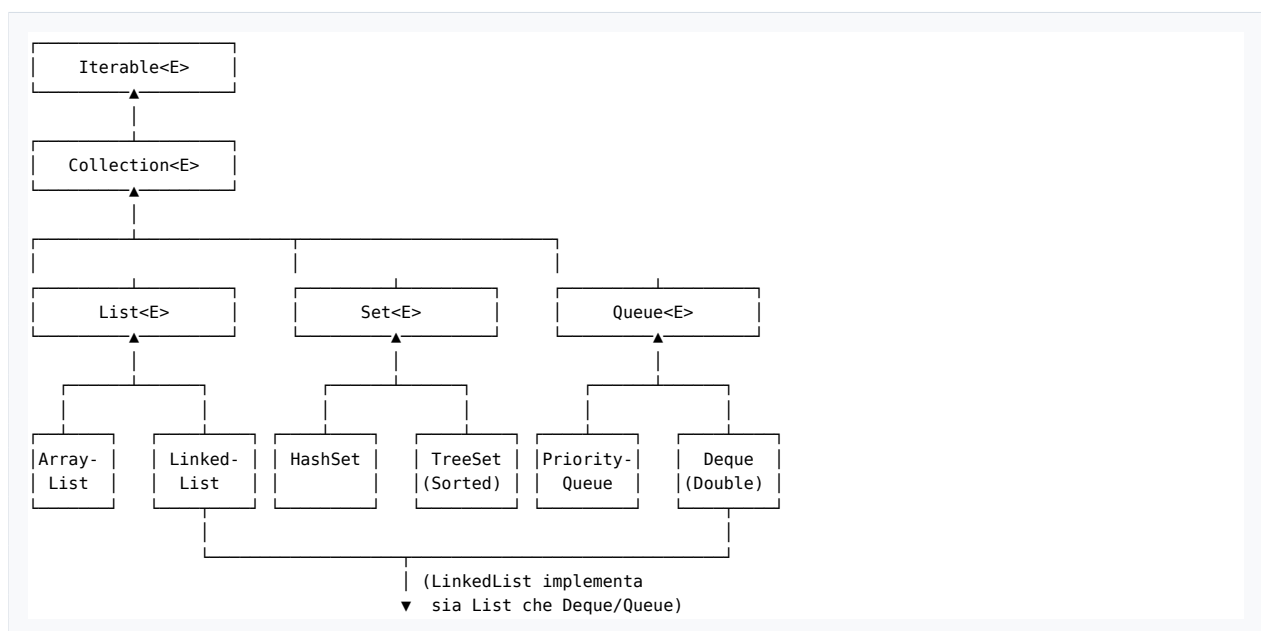
A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T16)

La lezione 16 è dedicata al **Java Collections Framework (JCF)** (`java.util`), una delle architetture cardine del linguaggio Java: un insieme unificato e standardizzato di interfacce, implementazioni concrete e algoritmi polimorfici per la gestione di collezioni di oggetti.

0.1.1 1.1 Architettura e Gerarchia delle Collezioni (Slide 3-4)

Tutte le collezioni di elementi singoli discendono dall'interfaccia radice `Iterable<E>`:



Nota di Studio

L'interfaccia `Map<K, V>` (dizionari chiave-valore) **NON estende** `Collection<E>`, poiché memorizza coppie (K, V) anziché singoli elementi. Tuttavia, fa parte integrante a pieno titolo del *Java Collections Framework*.

0.1.2 1.2 L'Interfaccia Radice `Collection<E>` (Slide 6-7)

Definisce il contratto comune a tutti i contenitori di elementi:

```

1 public interface Collection<E> extends Iterable<E> {
2     int size();
3     boolean isEmpty();
4     boolean contains(Object o);           // Ricerca basata su equals()
5     boolean containsAll(Collection<?> c);
6     boolean add(E e);                   // Aggiunge un elemento (restituisce false se
        rifiutato)
7     boolean addAll(Collection<? extends E> c); // Wildcard covariante!
8     boolean remove(Object o);
9     boolean removeAll(Collection<?> c);
10    void clear();
11    Object[] toArray();
12    <T> T[] toArray(T[] a);             // es. set.toArray(new Person[0])
13    Iterator<E> iterator();             // Ereditato da Iterable<E>
14 }
```

Interscambiabilità tra Collezioni (Slide 7) Tutte le implementazioni forniscono per convenzione un costruttore che accetta un'altra `Collection<? extends E>`, permettendo conversioni istantanee:

```

1 Collection<Person> list = new LinkedList<>();
2 list.add(new Person("Joe"));
3 list.add(new Person("Sam"));
4
5 // Conversione da List a Set (elimina al volo eventuali duplicati):
6 Collection<Person> set = new HashSet<>(list);
7
8 // Esportazione in Array tipizzato:
9 Person[] arrayP = set.toArray(new Person[0]); // Idioma standard Java
```

0.1.3 1.3 L'Interfaccia `List<E>` e le sue Implementazioni (Slide 8-9)

Una **Lista** è una sequenza ordinata di elementi che:

1. **Preserva l'ordine di inserimento.**
2. **Accetta elementi duplicati** (possono esistere più elementi e_1, e_2 tali che $e_1.equals(e_2)$ sia vero).
3. **Fornisce accesso posizionale indicizzato** tramite indice intero $[0 \dots size - 1]$:
 - * `E get(int index)`: legge l'elemento all'indice specificato.
 - * `E set(int index, E element)`: rimpiazza l'elemento alla posizione data.
 - * `void add(int index, E element)`: inserisce shiftando a destra gli elementi successivi.
 - * `E remove(int index)`: elimina shiftando a sinistra.
 - * `int indexOf(Object o)`: restituisce l'indice della prima occorrenza (o -1).
 - * `List<E> subList(int fromIndex, int toIndex)`: vista sulla porzione $[fromIndex, toIndex)$.

Confronto delle Implementazioni: `ArrayList` vs `LinkedList` (Slide 5 e 9)

Operazione	<code>ArrayList<E></code> (Array ridimensionabile)	<code>LinkedList<E></code> (Lista doppiamente concatenata)
Accesso per indice (<code>get(i)</code>)	$O(1)$ (accesso diretto in memoria contigua)	$O(n)$ (deve scorrere i nodi dal capo più vicino)
Inserimento in coda (<code>add(e)</code>)	$O(1)$ ammortizzato	$O(1)$
Inserimento in testa (<code>add(0, e)</code>)	$O(n)$ (deve copiare e shiftare l'array)	$O(1)$ (cambio puntatori dei nodi)
Cancellazione (<code>remove(i)</code>)	$O(n)$ (shift dei successivi)	$O(1)$ se si ha il riferimento al nodo, $O(n)$ per cercarlo
Overhead di memoria	Minimo (array contiguo con eventuale <i>capacity</i> vuota)	Alto (ogni nodo crea un oggetto con puntatori <i>prev</i> e <i>next</i>)
Funzionalità extra	<code>ensureCapacity(int minCapacity)</code>	Implementa anche <i>Queue</i> e <i>Deque</i> : <code>addFirst()</code> , <code>addLast()</code> , <code>pop()</code> , <code>push()</code>

0.1.4 1.4 L'Interfaccia `Set<E>` e le sue Implementazioni (Slide 10-16)

Un **Insieme (Set)** modella il concetto matematico di insieme: 1. **Nessun duplicato consentito**: non possono mai coesistere due elementi tali che `e1.equals(e2)` sia vero. 2. **Nessun accesso posizionale**: non esistono indici, non si può fare `get(i)`. 3. L'interfaccia `Set<E>` non aggiunge nuovi metodi rispetto a `Collection<E>`, ma ne restringe il contratto semantico.

1. `HashSet<E>` (Slide 11-14)

- **Struttura interna**: Mantiene internamente una tabella hash (`HashMap<E, Object>`).
- **Complessità**: Ricerca (`contains`), inserimento (`add`) e rimozione (`remove`) avvengono in **tempo costante medio** $O(1)$.
- **Ordine**: **Nessun ordine garantito**; l'ordine di scansione dell'iteratore può cambiare nel tempo se la tabella viene riallocata (*rehash*).
- **Vincolo Fondamentale (Slide 12-14)**: L'univocità si basa su `equals()` e `hashCode()`.
 - **La Regola Aurea**: “Se fai l'override di `equals()`, DEVI fare l'override anche di `hashCode()`, e viceversa”.
 - Proprietà vincolante: `a.equals(b)  $\Rightarrow$  a.hashCode() == b.hashCode()`.
 - Proprietà desiderabile (riduzione collisioni): `!a.equals(b)  $\Rightarrow$  preferibilmente a.hashCode() != b.hashCode()`.
 - Da Java 7 si usa l'utility di sistema: `Objects.hash(campo1, campo2, ...)`.

2. `TreeSet<E>` (Slide 15-16)

- **Struttura interna**: Albero binario di ricerca auto-bilanciante (**Red-Black Tree**).
- **Complessità**: `add`, `remove`, `contains` richiedono tempo logaritmico $O(\log n)$.
- **Ordinamento garantito**: Gli elementi sono **costantemente mantenuti ordinati**:
 - Ordinamento naturale: richiede che gli elementi implementino `Comparable<E>`.
 - Ordinamento esplicito: passando un `Comparator<E>` al costruttore: `new TreeSet<>(comparator)`.
- **Metodi speciali per insiemi ordinati**:
 - `E first()`: elemento minimo.
 - `E last()`: elemento massimo.
 - `TreeSet<E> subSet(E fromElement, E toElement)`: sottoinsieme nell'intervallo $[fromElement, toElement)$.

0.1.5 1.5 Mappe Associative: Map<K, V> (Slide 17-19)

Una mappa modella una funzione matematica $K \rightarrow V$ (tabella di associazioni chiave-valore):

- * Le **chiavi** (K) **sono univoche** (formano un $\text{Set}<K>$).
- * A ogni chiave è associato esattamente un valore (V). I valori possono ripetersi.
- * Metodi cardine:
 - * $V \text{ put}(K \text{ key}, V \text{ value})$: inserisce o rimpiazza l'associazione.
 - * $V \text{ get}(\text{Object key})$: restituisce il valore (o `null` se la chiave non esiste).
 - * `boolean containsKey(Object key)` / `boolean containsValue(Object value)`
 - * $\text{Set}<K> \text{ keySet}()$: restituisce la vista dell'insieme delle chiavi.
 - * $\text{Collection}<V> \text{ values}()$: restituisce la collezione dei valori.
 - * $\text{Set}<\text{Map.Entry}<K, V>> \text{ entrySet}()$: restituisce le coppie chiave-valore.
- * **HashMap<K, V>**: implementazione standard basata su hashing delle chiavi, con operazioni `put` e `get` in $O(1)$.

0.1.6 1.6 Iteratori e l'Errore ConcurrentModificationException (Slide 21-23)

Tutte le collezioni forniscono un `Iterator<E>` (`hasNext()`, `next()`, `remove()`). Il ciclo `for-each` di Java è uno zucchero sintattico compilato tramite `Iterator`:

```
1 for (Integer i : list) {
2     System.out.println(i);
3 }
```

La Trappola della Modifica Concorrente (*Fail-Fast Iterator*, Slide 23)

```
1 // CODICE ERRATO:
2 int count = 0;
3 for (Integer i : list) {
4     System.out.println(i);
5     if (count == 2)
6         list.remove(count); // RUNTIME ERROR: java.util.ConcurrentModificationException!
7     count++;
8 }
```

- **Perché crasha?**: Le collezioni Java mantengono internamente un contatore delle modifiche strutturali (`modCount`). Quando si invoca `list.remove()`, `modCount` viene incrementato. Al ciclo successivo, la chiamata implicita a `it.next()` scopre che la collezione è cambiata alle sue spalle (*fail-fast*) e lancia immediatamente `ConcurrentModificationException` per prevenire stati incoerenti.
- **La Soluzione Corretta (Slide 23)**: Usare esplicitamente l'iteratore e invocare il **suo** metodo `remove()`, che aggiorna coerentemente lo stato interno dell'iteratore:

```
1 for (Iterator<Integer> it = list.iterator(); it.hasNext(); ) {
2     Integer i = it.next();
3     if (count == 2)
4         it.remove(); // PERFETTAMENTE LEGALE E SICURO!
5     count++;
6 }
```

0.1.7 1.7 Algoritmi di Ordinamento: Comparable vs Comparator (Slide 24-29)

La classe di utilità `Collections` (`java.util.Collections`) fornisce due varianti sovraccaricate del metodo statico `sort`:

Variante 1: Ordinamento Naturale

```
1 public static <T extends Comparable<? super T>> void sort(List<T> list)
```

- **Perché <? super T> nella firma? (Slide 26 - Domanda Classica d'Esame):** Supponiamo di avere `class Student extends Person`. Se solo `Person` implementa `Comparable<Person>`, allora `Student` eredita il confronto basato su `Person`. Se la firma fosse `<T extends Comparable<T>>`, allora `Student` dovrebbe implementare tassativamente `Comparable<Student>`. Con `Comparable<? super T>`, una lista di `Student` può essere ordinata legalmente usando il `compareTo` della superclasse `Person` (`Comparable<Person>`), garantendo il massimo riuso polimorfico!

Variante 2: Ordinamento On-Demand con Comparator

```
1 public static <T> void sort(List<T> list, Comparator<? super T> cmp)
```

- Permette di definire criteri di ordinamento arbitrari senza toccare la classe sorgente, passando:
 1. Una classe dedicata (`class StudentComparator implements Comparator<Student>`).
 2. Una **classe anonima** (`new Comparator<Student>(){ ... }`).
 3. Un'espressione **Lambda** (`((s1, s2)-> s2.getMatricola()- s1.getMatricola())`).

0.2 2. Risccontro Pratico nel Codice (Lezione15/Lezione16)

Come osservato durante l'ispezione dei sorgenti: 1. Nella Lezione 15 il docente ha introdotto tutte le strutture viste nella Slide T16 all'interno di `MainDate.java`: `* List<Date> dateList = new ArrayList<>();` (con duplicato `AmericanDate(7,11,2025)` $\Rightarrow$ dimensione 4). `* Set<Date> dateSet = new HashSet<>();` (duplicato scartato $\Rightarrow$ dimensione 3). `* Map<String, FormattedDateConverter> converter = new HashMap<>();` * Ordinamento naturale: `Collections.sort(dateList);` * Ordinamento con `Comparator` via classe anonima e lambda su `DateInterval`. 2. Nella Lezione 16 il docente compie il passo architetturale successivo: separa la gestione degli errori creando il package `it.oop.exception` con eccezioni personalizzate (`IllegalDateException` e `OrderPairException`), preparando il terreno per la teoria della Slide T17.

Dimmi “vai” per procedere con il **Blocco 16** (Lezione 17 / Slide T17 — *Error Handling with Exceptions*, gerarchia `Throwable/Exception/RuntimeException`, eccezioni controllate vs non controllate, costruito `try-catch-finally`, `try-with-resources` e il codice di `Lezione16/MavenDate`).